

1 6/4: Gradle+Kotlin のセットアップ

```
gradle --version
```

で version の確認を行った, 8.x.x だったため,

```
gradle init
./gradlew wrapper --gradle-version 9.5.1
```

を行った.

```
./gradlew run
```

で Hello World! の出力が得られた. これは, app/src/main/kotlin/org/example/App.kt 内の main() 関数が実行されたものである. 実行される関数は, app/src/build.gradle.kts 内の

```
application {
    mainClass = "org.example.AppKt"
}
```

の設定によって決まった.

.../org/example/SerializationDemo.kt を作成して, ライブラリの有効化を試した. gradle/libs.versions.toml の [plugins] に追加して, build.gradle.kts の plugins に alias() で追加する. 同じく [libraries] に追加して, build.gradle.kts の dependencies に implementation() で追加した.

2 6/12: 依存関係の試行

editor を IntelliJ に変えて, app/build.gradle.kts にエラーが生じることを解決しようとしたが, できなかった. Hello.kt(App.kt から rename) を実行しても問題なく動くので editor 側の表示だけの問題だろう.

2.1 Serialization を用いたプログラム

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class Task(
    // 省略
)
```

```

fun main() {
    val task = Task(
        id = 0,
        name = "Learn Serialization"
    )
    val json = Json.encodeToString(task)
    println("Encoded: $json")

    val decoded = Json.decodeFromString<Task>(json)
    println("Decoded: $decoded")
}

```

出力結果は,

```

Encoded: {"id":0,"name":"Learn Serialization"}
Decoded: Task(id=0, name=Learn Serialization, done=false)

```

因みに, ./gradlew dependencies は, No configurations を返した.

上のプログラムでは, プログラム実行中のメモリに各 task が保持されている. project directory 内に json ファイルを作成し, そのファイルへの書き込み/書き出しを実装してみた.

```

import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
import java.io.File

// annotation, data class の定義は省略

fun main() {
    val file = File("tasks.json")
    val tasks = if (file.exists()) {
        Json.decodeFromString<MutableList<Task>>(file.readText())
    } else {
        mutableListOf()
    }

    val addedTask = Task(
        id = 0,
        name = "Learn Serialization"
    )
}

```

```

        tasks.add(addedTask)

        file.writeText(Json.encodeToString(tasks))
        println(tasks)
    }

```

出力は,

```
[Task(id=0, name=Learn Serialization, done=false)]
```

前回と出力形式が変わっている。前は `println(task)` (`task` は `Task` 型), 今回は json ファイルを書き出してそのまま出力している。 `Json.decodeFromString<MutableList<Task>>` から, `Task` 型のリストとして出力されたと考えられる。また, `file=File("task.json")` はファイル名ではなく path 名を表している。この path が `.../app/src` の下から始まっていると予想して, path を `"main/kotlin/tasks.json"` とするとエラーが出た。続いて, `"src/main/kotlin/tasks.json"` とすると正常に動作した。よってこの部分の path は `.../app` の下から始まっていることが分かった。

続いて, 書き込み/書き出しを分けて実装して, `build.gradle.kts/application` での `mainClass` の切り替えで `tasks.json` を操作した。 `.../example/AddTask.kt` および `ListTasks.kt` と, `.../example/model/Task.kt` を追加した。まず, `Task.kt` には data class の定義を書いた。このファイルにだけ annotation `@Serializable` を書く。

```

import kotlinx.serialization.Serializable

@Serializable
data class Task(
    val id: Int,
    val name: String,
    val done: Boolean = false
)

```

`addTask.kt` には, ファイルの発見/作成と書き込みを実装する。

```

import kotlinx.serialization.json.Json
import java.io.File
import org.example.model.Task

fun main() {
    val file = File("src/main/kotlin/data/tasks.json")
    // 変更できる (mutable) リストとして task.json を書き出し
    val tasks = if (file.exists()) {
        Json.decodeFromString<MutableList<Task>>(file.readText())
    } else {
        mutableListOf()
    }
}

```

```

    } else {
        mutableListOf()
    }

    val addedTask = Task(
        id = 1,
        name = "Clean the Room"
    )
    // リストの変更
    tasks.add(addedTask)
    // リスト → json ファイル
    file.writeText(Json.encodeToString(tasks))

    println("Added. Total: ${tasks.size}")
}

```

ListTasks.kt には、ファイルの発見と書き出しを実装する。

```

import kotlinx.serialization.json.Json
import org.example.model.Task
import java.io.File

fun main() {
    val file = File("src/main/kotlin/data/tasks.json")
    if (!file.exists()) {
        println("No tasks found.")
        return
    }

    // 読みだけの (mutable でない) リストとして tasks.json を書き出し
    val tasks = Json.decodeFromString<List<Task>>(file.readText())
    // tasks の各要素を it として受ける
    tasks.forEach { println("[${it.id}] ${it.name} - done: ${it.done}") }
}

```